

Casos de Teste de Segurança - SIBiSC

Entrega final - Rafael Feltrim - N° USP: 15942812
04_sibisc_projeto_grupo\qa\casos_de_teste_seguranca.md

Casos de Teste de Segurança - SIBiSC

Objetivo

Definir roteiros de teste de segurança para o MVP do SIBiSC, focando em proteção de dados, integridade de acesso e conformidade das políticas de segurança do Supabase (RLS - Row Level Security).

Escopo

Os testes cobrem as seguintes áreas:

1. Injeção de SQL/NoSQL via Supabase
2. Proteção de rotas privadas (Bibliotecário / Curador)
3. Políticas de RLS (Row Level Security)
4. Validação de entrada (inputs do usuário)
5. Exposição de dados sensíveis

Convenções

- ID do cenário: SEC-[área]-[número]
- Formato: Given-When-Then
- Severidade: Crítica, Alta, Média, Baixa
- OWASP: Referência à categoria OWASP Top 10 quando aplicável

1. Injeção de SQL / NoSQL

> OWASP A03:2021 - Injection

SEC-INJ-001 — Tentativa de SQL injection no campo de busca de livros

- Severidade: Crítica
- Componente: CatalogPage / SearchField

Given que o estudante está na página do catálogo (/catalogo)
When digita no campo de busca o payload: ' OR '1'='1
Then a busca NÃO retorna todos os registros do banco
And o input é tratado como texto literal
And nenhum erro de SQL é exposto na interface
And o console do navegador não exibe queries SQL brutas

SEC-INJ-002 — Tentativa de SQL injection via parâmetro de URL

- Severidade: Crítica
- Componente: Rotas dinâmicas (/:newsId, /:eventId, /:bookId)

Given que um atacante acessa a URL /catalogo/1; DROP TABLE books;--
When a aplicação processa o parâmetro da rota
Then o parâmetro é tratado como identificador inválido
And a página exibe um erro amigável ou redireciona para 404
And nenhuma operação destrutiva é executada no banco de dados

SEC-INJ-003 — Payload de injeção em campos de formulário

- Severidade: Crítica
- Componente: Qualquer formulário com submissão ao Supabase

Given que um usuário autenticado (Bibliotecário) preenche um formulário de cadastro
When insere no campo de título: '; DELETE FROM events WHERE '1'='1

Then o Supabase trata o valor como string literal via parameterized query
And o registro é criado com o texto exato digitado (sem execução de SQL)
And nenhum registro existente é afetado

SEC-INJ-004 — Tentativa de NoSQL injection via JSON malformatado

- Severidade: Alta
- Componente: Serviços que enviam dados ao Supabase

Given que um atacante envia um payload JSON manipulado via DevTools
When o payload contém operadores como {"\$gt": ""} ou {"\$ne": null}
Then o Supabase (PostgreSQL) rejeita operadores NoSQL
And a requisição retorna erro 400 ou é ignorada
And nenhum dado é vazado ou modificado

2. Proteção de Rotas Privadas

> OWASP A01:2021 - Broken Access Control

SEC-AUTH-001 — Acesso a área administrativa sem autenticação

- Severidade: Crítica
- Componente: Rotas de operação interna (Bibliotecário/Curador)

Given que um usuário não autenticado tenta acessar uma rota administrativa
When navega diretamente para /admin ou /gerenciar (se existir)
Then o acesso é bloqueado
And o usuário é redirecionado para a página de login ou para a Home
And nenhum dado administrativo é exibido antes do redirecionamento

SEC-AUTH-002 — Token de sessão expirado

- Severidade: Alta
- Componente: Supabase Auth / supabaseClient.js

Given que um Bibliotecário está logado com uma sessão válida
When o token JWT expira (tempo de expiração do Supabase)
Then as próximas requisições autenticadas são rejeitadas com 401
And o usuário é notificado sobre a expiração
And é redirecionado para a tela de login
And nenhuma operação de escrita é executada com token expirado

SEC-AUTH-003 — Tentativa de escalar privilégios

- Severidade: Crítica
- Componente: Supabase RLS + API

Given que um usuário autenticado com perfil "Cidadão" (leitura pública)
When tenta executar operações de escrita (INSERT, UPDATE, DELETE) via API do Supabase
Then todas as operações são bloqueadas pelas políticas de RLS
And o Supabase retorna erro 403 (Forbidden) ou resultado vazio
And o log do servidor registra a tentativa

SEC-AUTH-004 — Manipulação de JWT no lado do cliente

- Severidade: Crítica
- Componente: Supabase Auth

Given que um atacante intercepta seu próprio JWT válido
When modifica o payload do token para alterar o role (ex: de "anon" para "service_role")
Then o Supabase rejeita o token por assinatura inválida
And a requisição retorna erro 401 (Unauthorized)
And nenhum acesso privilegiado é concedido

3. Políticas de RLS (Row Level Security)

> OWASP A01:2021 - Broken Access Control

SEC-RLS-001 — Leitura pública de notícias (anon)

- Severidade: Alta
- Componente: Tabela notícias no Supabase

Given que um usuário anônimo (não autenticado) acessa a API do Supabase
When faz uma requisição SELECT na tabela de notícias
Then apenas notícias com status "publicado" são retornadas
And notícias em rascunho ou revisão NÃO são visíveis
And campos internos (notas do curador, logs de edição) NÃO são expostos

SEC-RLS-002 — Leitura pública de eventos (anon)

- Severidade: Alta
- Componente: Tabela eventos no Supabase

Given que um usuário anônimo acessa a API do Supabase
When faz uma requisição SELECT na tabela de eventos
Then apenas eventos com status "publicado" são retornados
And eventos cancelados ou em rascunho NÃO são visíveis
And informações operacionais internas NÃO são expostas

SEC-RLS-003 — Leitura pública de acervo (anon)

- Severidade: Alta
- Componente: Tabelas livros e inventario no Supabase

Given que um usuário anônimo acessa a API do Supabase
When faz uma requisição SELECT nas tabelas do acervo
Then os dados públicos (título, autor, ISBN, disponibilidade) são retornados
And dados operacionais (custo, fornecedor, notas internas) NÃO são expostos

SEC-RLS-004 — Bloqueio de escrita para usuários anônimos

- Severidade: Crítica
- Componente: Todas as tabelas do Supabase

Given que um usuário anônimo (role: anon) tenta acessar a API
When envia requisições INSERT, UPDATE ou DELETE em qualquer tabela
Then todas as operações são bloqueadas pelo RLS
And o Supabase retorna erro ou resultado vazio
And nenhum dado é modificado

SEC-RLS-005 — Escrita permitida apenas para roles autorizados

- Severidade: Crítica
- Componente: Tabelas de notícias e eventos no Supabase

Given que um Bibliotecário autenticado (role: bibliotecario) acessa a API
When envia uma requisição INSERT para cadastrar uma notícia ou evento
Then a operação é permitida pelo RLS
And o registro é criado com o user_id do bibliotecário
And a operação é auditável (created_by, created_at)

SEC-RLS-006 — Isolamento de dados entre roles

- Severidade: Alta
- Componente: Supabase RLS policies

Given que existem dois bibliotecários autenticados (A e B)
When o Bibliotecário A tenta editar um registro criado pelo Bibliotecário B
Then a política de RLS define se a edição é permitida (se cross-editing é habilitado) ou bloqueada
And a decisão é explícita na política, não implícita por ausência de regra

4. Validação de Entrada (Input Validation)

> OWASP A03:2021 - Injection e OWASP A07:2021 - XSS

SEC-XSS-001 — Tentativa de XSS no campo de busca

- Severidade: Alta
- Componente: SearchField / CatalogPage

Given que o estudante está na página do catálogo
When digita no campo de busca: `<script>alert('xss')</script>`
Then o texto é tratado como string literal
And NENHUM script é executado no navegador
And o texto é exibido como texto puro nos resultados (se houver match)
And o React escapa automaticamente o conteúdo via JSX

SEC-XSS-002 — Conteúdo malicioso em dados renderizados

- Severidade: Alta
- Componente: Páginas de detalhe (NotíciasDetail, EventDetail, BookDetail)

Given que um registro no banco contém HTML malicioso no campo de descrição
(ex: `<img src=x onerror=alert('xss')">`)
When a página de detalhe renderiza esse conteúdo
Then o HTML malicioso é escapado pelo React
And nenhum evento JavaScript é disparado
And o conteúdo é exibido como texto plano

SEC-VAL-001 — Limite de caracteres em campos de busca

- Severidade: Média
- Componente: SearchField

Given que o estudante está no campo de busca
When digita uma string com mais de 500 caracteres
Then o campo limita a entrada ou trunca o valor
And a requisição ao backend não contém payloads excessivamente grandes

5. Exposição de Dados Sensíveis

> OWASP A02:2021 - Cryptographic Failures e OWASP A05:2021 - Security Misconfiguration

SEC-EXP-001 — Chaves de API não expostas no client-side

- Severidade: Crítica
- Componente: supabaseClient.js / .env

Given que a aplicação está em produção
When um atacante inspeciona o código-fonte no navegador (DevTools > Sources)
Then a SUPABASE_URL e a PUBLISHABLE_KEY (anon key) podem ser visíveis (isso é esperado)
And a SERVICE_ROLE_KEY NÃO está presente no código client-side
And nenhuma chave de admin, secret ou senha do banco está exposta

SEC-EXP-002 — Arquivo .env não commitado no repositório

- Severidade: Crítica
- Componente: Repositório Git

Given que o repositório está no GitHub
When se verifica o conteúdo do .gitignore
Then o arquivo .env está listado no .gitignore
And nenhum arquivo .env existe no histórico de commits
And apenas o .env.example (sem valores reais) está versionado

SEC-EXP-003 — Headers de segurança HTTP

- Severidade: Média
- Componente: Servidor de produção (Vite preview ou hosting)

Given que a aplicação está servida em produção
When se inspecionam os headers HTTP de resposta
Then os seguintes headers de segurança estão presentes (quando configuráveis):

- X-Content-Type-Options: nosniff
 - X-Frame-Options: DENY ou SAMEORIGIN
 - Referrer-Policy: strict-origin-when-cross-origin
- And o Content-Security-Policy é definido quando possível

Ferramentas Recomendadas para Execução

Ferramenta / Finalidade / Tipo

- Ferramenta: Supabase Dashboard > SQL Editor; Finalidade: Testar queries e políticas RLS diretamente; Tipo: Manual
- Ferramenta: DevTools > Console + Network; Finalidade: Inspeccionar tokens, headers e requisições; Tipo: Manual
- Ferramenta: OWASP ZAP; Finalidade: Scanner automatizado de vulnerabilidades web; Tipo: Automatizado
- Ferramenta: Burp Suite Community; Finalidade: Interceptação e manipulação de requisições HTTP; Tipo: Manual
- Ferramenta: ESLint Security Plugin; Finalidade: Análise estática de código para padrões inseguros; Tipo: Automatizado
- Ferramenta: npm audit; Finalidade: Verificação de dependências com vulnerabilidades conhecidas; Tipo: Automatizado

Referências

- OWASP Top 10 (2021): <https://owasp.org/www-project-top-ten/>
- Supabase Row Level Security: <https://supabase.com/docs/guides/auth/row-level-security>
- Supabase Auth: <https://supabase.com/docs/guides/auth>
- React XSS Prevention: <https://legacy.reactjs.org/docs/introducing-jsx.html#jsx-prevents-injection-attacks>